Datum: 12. 10. 2026.

Struktura projekta

Projekt je razdeljen na obvezne in neobvezne zahteve zahteve. Izpolniti morate vse obvezne zahteve in vsaj 50% skupnih zahtev. Zahteve se preverjajo med zagovorom projekta in se ocenjujejo glede na obseg in kakovost implementacije. Minimalne zahteve so določene za vsakega člana skupine in se smiselno prilagodijo glede na število članov v skupini, kjer je to mogoče (npr. če je minimalna zahteva razvoj dveh mikrostoritev, je za tričlansko skupino minimalna zahteva šest mikrostoritev).

Končni izdelek

Pri vajah se kot končni izdelek ocenjujejo repozitoriji Github in poročilo o projektu. Več informacij o pripravi poročila bo v ločenem dokumentu.

Zahteve projekta

V preostanku dokumenta so predstavljene zahteve projekta in pripadajoče število točk za projekt. Najprej so navedene tabele, nato daljši opisi.

Konfiguracijske zahteve

Indeks	Ime	Opis	Obvezno	Točke (%)
1	Repozitorij	Ustvarite repozitorij za projekt z uporabo git.	Da	1
2	Mikrostoritve in Cloud-native aplikacija	Nastavite razvojno okolje in definirajte strukturo projekta, zasnujte in implementirajte aplikacijo z uporabo arhitekture mikrostoritev ter jih namestite na Kubernetes gručo.	Da	4
3	Podatkovna baza za vsako mikrostoritev (database-per-service)	Vsaki mikrostoritvi, ki potrebuje trajno shranjevanje podatkov, zagotovite lastno podatkovno bazo.	Da	2

Indeks	Ime	Opis	Obvezno	Točke (%)
4	Dokumentacija	Ustvarite tehnično dokumentacijo za projekt.	Da	2
5	Dokumentacija API	Ustvarite dokumentacijo za API.	Ne	3
6	Cevovod CI/CD	Uvedite CI/CD (nenehno integracijo in nenehno dobavo).	Ne	5
7	Helm charts	Uporabi Helm Charts za upravljanje namestitev Kubernetes.	Ne	4
8	Namestitev v oblak	Namestite aplikacijo v oblačnem okolju.	Da	5
Skupaj				**26**

Zahteve o funkcionalnosti

Indeks	Ime	Opis	Obvezno	Točke (%)
9	Zunanji API	V aplikacijo integrirajte vsaj en zunanji API.	Ne	3
10	Večnajemništvo (multitenancy)	Implementirajte večnajemniško arhitekturo.	Ne	5
11	Preverjanje zdravja	Implementirajte preverjanja zdravja za mikrostoritve.	Ne	4
12	Event sourcing in CQRS	Implementirajte vzorce *Event Sourcing* in CQRS.	Ne	5
13	Izolacija in toleranca napak	Zagotovite izolacijo mikrostoritev in odpornost na napake.	Ne	5

Indeks	Ime	Opis	Obvezno	Točke (%)
14	Upravljanje s konfiguracijo	Implementirajte koncepte za upravljanje konfiguracije.	Ne	4
15	Grafični vmesnik	Razvijte grafični uporabniški vmesnik (GUI) za aplikacijo.	Da	4
Skupaj				**30**

Tehnološke zahteve

Indeks	Ime	Opis	Obvezno	Točke (%)
16	Serverless funkcija	Razvijte in namestite *serverless* funkcijo za del aplikacije.	Ne	5
17	GraphQL in gRPC	Uporabite GraphQL in gRPC za komunikacijo med storitvami.	Ne	4
18	Sporočilni sistemi	Implementirajte sporočilni sistem za komunikacijo med storitvami.	Ne	5
19	Centralizirano beleženje dnevnikov	Implementirajte centraliziran sistem za beleženje in analizo dnevnikov.	Ne	5
20	Zbiranje metrik	Implementirajte sistem za zbiranje metrik delovanja aplikacije.	Ne	5
Skupaj				**24**

Zahteve o oddaji projekta

Indeks	Ime	Opis	Obvezno	Točke (%)
21	Vmesna predstavitev	Kratek javni *pitch* projekta na vajah s predstavitvijo arhitekturne zasnove in že razvitih delov (neobvezno).	Ne	10
22	Oddaja projekta	Oddaja kode in poročila na spletni učilnici.	Da	10
Skupaj				**20**

Zahteve za dodatne točke

Poleg zgornjih zahtev lahko v rešitev vključite tudi spodnje koncepte za dodatne točke. Te točke se ne štejejo v kvoto za opravljene vaje, najvišja možna končna ocena pa ostaja 100%.

Indeks	Ime	Opis	Obvezno	Točke (%)
23	Terraform	Uporabite Terraform za avtomatizacijo namestitve infrastrukture v oblak.	Ne	3
24	API Gateway	Implementirajte API Gateway za centralizirano upravljanje dostopa do mikrostoritev.	Ne	4
25	Ingress Controller	Ustvarite Ingress Controller za upravljanje dostopa do storitev v Kubernetes.	Ne	4
26	IAM, OAuth2, OIDC	Uporabite upravljanje identitete in dostopa (IAM) sistem za upravljanje identitet in dostopa.	Ne	4
Skupaj				**15**

Podrobnosti zahtev

1. Repozitorij

Ustvarite repozitorij za projekt z uporabo Git.

Obvezno: Da | **Točke:** 1

Minimalne zahteve:

- Nastavite Git repozitorij na platformi GitHub (lahko tudi GitLab ali Bitbucket).
- Vključite datoteko README z informacijami o projektu (kratek opis, navodila za namestitev itd.).
- Uvedite strategijo razvejitve (npr. main, dev).

2. Mikrostoritve in Cloud-native aplikacija

Nastavite razvojno okolje in definirajte strukturo projekta, zasnujte in implementirajte aplikacijo z uporabo arhitekture mikrostoritev ter jih namestite na gručo Kubernetes.

Obvezno: Da | **Točke: 4**

Minimalne zahteve:

- Uporabite IDE (npr. Visual Studio Code, IntelliJ IDEA).
- Nastavite osnovno strukturo projekta v skladu z najboljšimi praksami (npr. direktoriji za izvorno kodo, teste in konfiguracijske datoteke).
- Inicializirajte razvojno okolje z vsemi potrebnimi datotekami.
- Dokumentirajte navodila za nastavitev za druge razvijalce.
- Razdelite aplikacijo na vsaj dve mikrostoritvi poleg GUI, kjer je vsaka odgovorna za določeno funkcionalnost. Za enočlansko ekipo je minimum torej GUI + dve mikrostoritvi.
- Zagotovite, da mikrostoritve komunicirajo prek HTTP ali sporočilnega protokola.
- Napišite manifeste Kubernetes (datoteke YAML) za namestitev mikrostoritev.
- Uporabite funkcionalnosti Kubernetes, kot so storitve, namestitve in podi za orkestracijo.
- Upoštevajte *cloud-native* koncepte.
- Upoštevajte koncepte *clean architecture*.

3. Podatkovna baza za vsako mikrostoritev (database-per-service)

Vsaki mikrostoritvi, ki potrebuje trajno shranjevanje podatkov, zagotovite lastno podatkovno bazo (vzorec database-per-service).

Obvezno: Da | **Točke: 2**

Minimalne zahteve:

- Uporabite relacijsko (npr. PostgreSQL, MySQL) ali NoSQL (npr. MongoDB) bazo podatkov.
- Vsaka mikrostoritev, ki potrebuje trajno shranjevanje podatkov, ima svojo lastno podatkovno bazo (mikrostoritve, ki podatkov ne hranijo, baze ne potrebujejo).
- Mikrostoritve ne dostopajo neposredno do podatkovnih baz drugih mikrostoritev. Do njihovih podatkov dostopajo le prek API-jev ali sporočil.
- Zagotovite, da mikrostoritve v svoje baze shranjujejo in iz njih berejo podatke.
- V podatkovne baze dodajte nekaj tabel in podatkov.

4. Dokumentacija

Ustvarite tehnično dokumentacijo za projekt.

Obvezno: Da | **Točke:** 2

Minimalne zahteve:

- Vključite vse relevantne informacije o aplikaciji glede na primere dobrih praks.
- Uporabite Markdown ali druga orodja za dokumentacijo za jasno organizacijo vsebine.

5. Dokumentacija API

Ustvarite dokumentacijo za API.

Obvezno: Ne | **Točke:** 3

Minimalne zahteve:

- Uporabite orodja OpenAPI ali Postman za dokumentiranje končnih točk API.
- Vključite podrobnosti o formatih zahtev/odgovorov, primerih tovorov in sporočilih o napakah za vsako mikrostoritev.

6. Cevovod CI/CD

Uvedite CI/CD (angl. *continuous integration and continuous deployment*).

Obvezno: Ne | **Točke:** 5

Minimalne zahteve:

- Uporabite orodje CI/CD za avtomatizacijo gradnje in testiranja. Priporočamo GitHub Actions, lahko tudi npr. GitLab CI, Jenkins.
- Zagotovite, da se koda samodejno testira in namesti na gručo Kubernetes ob spremembah v glavni veji.

7. Helm charts

Uporabite Helm Charts za upravljanje namestitev Kubernetes.

Obvezno: Ne | **Točke:** 4

Minimalne zahteve:

- Ustvarite Helm Charts za mikrostoritve.
- Parametrizirajte konfiguracije za različna okolja. Okolja naj vsebujejo vsaj razvoj (angl. *development/dev*) in produkcijo (angl. *production/prod*).
- Namestite in posodobite aplikacije v Kubernetes okolju s Helm Charts.

8. Namestitev v oblak

Namestite aplikacijo v oblačnem okolju.

Obvezno: Da | **Točke:** 5

Minimalne zahteve:

- Uporabite oblačno platformo. Priporočamo Microsoft Azure, lahko pa tudi AWS, ali Google Cloud.
- Zagotovite, da sta gruča Kubernetes in aplikacija javno dostopni.

9. Zunanji API

V aplikacijo integrirajte vsaj en zunanji API.

Obvezno: Ne | **Točke:** 3

Minimalne zahteve:

- Uporabite zunanji API znotraj ene izmed mikrostoritev.
- Ustrezno izvedite potrditev verodostojnosti (angl. *authentication*).

10. Večnajemništvo (multitenancy)

Implementirajte večnajemniško arhitekturo.

Obvezno: Ne | **Točke:** 5

Minimalne zahteve:

- Načrtujte in implementirajte večnajemništvo na nivoju mikrostoritev, Kubernetes in podatkovne baze.
- Prikažite delovanje z več najemniki.

11. Preverjanje zdravja

Implementirajte preverjanja zdravja za mikrostoritve.

Obvezno: Ne | **Točke:** 4

Minimalne zahteve:

- Ustvarite končne točke za preverjanje zdravja, ki vrnejo stanje vsake mikrostoritve.
- Zagotovite, da lahko Kubernetes spremlja te končne točke za zdravje storitve.

12. Event sourcing in CQRS

Implementirajte vzorce *Event Sourcing* in CQRS.

Obvezno: Ne | **Točke:** 5

Minimalne zahteve:

- Implementirajte *Event Sourcing* za sledenje sprememb podatkov.
- Uporabite CQRS za ločene modele branja in pisanja.
- Demonstrirajte, kako se lahko ponovno predvajajo dogodki za obnovo stanja.

13. Izolacija in toleranca napak

Zagotovite izolacijo mikrostoritev in odpornost na napake.

Obvezno: Ne | **Točke:** 5

Minimalne zahteve:

- Implementirajte mehanizme za toleranco napak, kot so Circuit Breaker in Retry.
- Prikažite, kako sistem ohranja delovanje tudi ob napakah ene storitve.
- Pojasnite, kako izolacija storitev zmanjšuje vpliv napak na celoten sistem.

14. Upravljanje s konfiguracijo

Implementirajte koncepte za upravljanje konfiguracije.

Obvezno: Ne | **Točke:** 4

Minimalne zahteve:

- Identificirajte vse konfiguracijske parametre vsake mikrostoritve (npr. nizi za povezavo s podatkovno bazo, URL-ji drugih storitev in zunanjih API-jev, poverilnice in ključi API, vrata, zastavice funkcionalnosti (angl. *feature flags*), nivoji beleženja) in jih navedite v dokumentaciji, skupaj z namenom in privzeto vrednostjo vsakega parametra.

- Ločite implementacijo in konfiguracijo. Konfiguracijo naj bo mogoče spreminjati brez ponovnega prevajanja in nameščanja mikrostoritve.
- Predvidite in uvedite eno uporabo različnih virov konfiguracije, npr. konfiguracijske datoteke, okoljske spremenljivke, konfiguracijski strežnik...

15. Grafični vmesnik

Razvijte grafični uporabniški vmesnik (angl. *graphical user interface, GUI*) za aplikacijo.

Obvezno: Da | **Točke:** 4

Minimalne zahteve:
- Uporabite ogrodje (npr. React, Angular, Vue.js) za izdelavo GUI ali razvijte mobilno aplikacijo.
- Implementirajte vsaj 2 podstrani.
- Zagotovite, da sprednji del komunicira z mikrostoritvami prek REST API-jev.

16. Serverless funkcija

Razvijte in namestite *serverless* funkcijo za del aplikacije.

Obvezno: Ne | **Točke:** 5

Minimalne zahteve:
- Uporabite Azure Functions, lahko tudi AWS Lambda ali Google Cloud Functions.
- Implementirajte funkcijo, ki se proži na podlagi dogodkov (npr. zahtevek HTTP).
- Opazujte delovanje funkcije in prilagajajte zmogljivost na podlagi obremenitev.

17. GraphQL in gRPC

Uporabite GraphQL in gRPC za komunikacijo med storitvami.

Obvezno: Ne | **Točke:** 4

Minimalne zahteve:
- Implementirajte GraphQL za poizvedovanje po podatkih v eni mikrostoritvi.
- Uporabite gRPC za komunikacijo med dvema storitvama.
- Prikažite uporabo obeh tehnologij v aplikaciji.

18. Sporočilni sistemi

Implementirajte sporočilni sistem za komunikacijo med storitvami.

Obvezno: Ne | **Točke:** 5

Minimalne zahteve:

- Uporabite sporočilnega posrednika (npr. RabbitMQ, Kafka, NATS Jetstream) za asinhrono komunikacijo med mikrostoritvami.
- Zagotovite, da vsaj dve mikrostoritvi komunicirata prek sporočilnega sistema.

19. Centralizirano beleženje dnevnikov

Implementirajte centraliziran sistem za beleženje in analizo dnevnikov.

Obvezno: Ne | **Točke:** 5

Minimalne zahteve:

- Uporabite orodja, kot so ELK Stack ali Fluentd za zbiranje dnevnikov.
- Centralizirajte dnevnike vseh storitev.
- Nastavite opozorila na podlagi zaznanih vzorcev v dnevnikih.

20. Zbiranje metrik

Implementirajte sistem za zbiranje metrik delovanja aplikacije.

Obvezno: Ne | **Točke:** 5

Minimalne zahteve:

- Uporabite Prometheus, Grafana ali podobno orodje za zbiranje in vizualizacijo metrik.
- Spremljajte ključne metrike aplikacije, kot so odzivnost in obremenitev.
- Prikažite uporabo zbranih metrik za optimizacijo delovanja aplikacije.

21. Vmesna predstavitev

Neobvezna vmesna predstavitev: kratek javni *pitch* projekta na vajah, predstavitev arhitekturne zasnove in že razvitih delov. Več informacij bo v ločenem dokumentu.

Obvezno: Ne | **Točke:** 10

22. Oddaja projekta

Oddaja kode in poročila na spletni učilnici. Več informacij bo v ločenem dokumentu.

Obvezno: Da | **Točke:** 10

23. Terraform

Uporabite Terraform za avtomatizacijo namestitve infrastrukture v oblak.

Obvezno: Ne (dodatne točke) | **Točke:** 3

Minimalne zahteve:

- Ustvarite Terraform konfiguracijske datoteke za definiranje potrebnih virov (npr. virtualni stroji, omrežja, baze podatkov).
- Uporabite module za ponovno uporabo konfiguracij.
- Implementirajte Terraform *plan* in *apply* za izpeljavo sprememb infrastrukture.

24. API Gateway

Implementirajte API Gateway za centralizirano upravljanje dostopa do mikrostoritev.

Obvezno: Ne (dodatne točke) | **Točke:** 4

Minimalne zahteve:

- Uporabite API Gateway (npr. Kong, AWS API Gateway, Istio) za usmerjanje API klicev.
- Ustrezno nastavite potrditev pristnosti (angl. *authentication*) in avtorizacijo (angl. *authorization*) preko API Gateway.
- Implementirajte enotno točko dostopa za različne API-je.
- Prikažite delovanje API Gateway z več mikrostoritvami.

25. Ingress Controller

Ustvarite Ingress Controller za upravljanje dostopa do storitev v Kubernetes.

Obvezno: Ne (dodatne točke) | **Točke:** 4

Minimalne zahteve:

- Namestite Ingress Controller (npr. NGINX, Traefik) v svoje Kubernetes okolje.
- Nastavite pravila Ingress za usmerjanje prometa do ustreznih storitev.
- Implementirajte TLS/SSL za varno komunikacijo.
- Preverite delovanje Ingress Controllerja z dostopom do storitev preko URL-jev.

26. IAM, OAuth2, OIDC

Uporabite upravljanje identitete in dostopa (IAM) sistem za upravljanje identitet in dostopa.

Obvezno: Ne (dodatne točke) | **Točke:** 4

Minimalne zahteve:

- Implementirajte IAM rešitev za upravljanje uporabniških dostopov. Priporočamo Azure AD, lahko tudi AWS IAM, Keycloak.
- Uporabite OAuth2 za avtorizacijo dostopa do API-jev.

- Implementirajte OpenID Connect (OIDC) za avtorizacijo in pridobivanje informacij o uporabnikih.
- Preverite delovanje s prijavo uporabnika in dodelitvijo pravic dostopa do aplikacije.